

# Lekcja 07 - Data contracts i schema drift

2026-08-17

Data Engineering Course

## Kontrakty danych i schema drift

```
surowy payload -> walidacja ->  
zaakceptowane/odrzucone -> metryki jakości
```

Cel: zatrzymać zły input zanim zniszczy KPI lub zafałszuje decyzję biznesową.

Dane są nie tylko „plikiem z wejściem”. Dane są produktem, który ktoś będzie czytał i na którym ktoś podejmie decyzję.

Kontrakt danych to jasna umowa między producentem a konsumentem:

```
wymagane pola  
typy pól  
dozwolone wartości  
ograniczenia zakresu  
polityka odrzucania
```

Jeśli tej umowy nie ma, pipeline jedynie „przepuszcza dane”. Nie gwarantuje jakości.

## ■ Dlaczego to jest ważne

Dashboard pokazuje przychody, statusy i marżę. Jeśli surowy payload ma dziwne pola, zły format albo ciche zmiany, biznes nie wie, czy dane są użyteczne.

```
brak złego kodu transformacji  
to zły input i brak kontraktu
```

Ten problem jest dużo droższy niż błąd w jednym warunku. Dotyczy zaufania do danych na całym downstream.

## ■ Czym jest kontrakt danych

Kontrakt danych to nie dokument techniczny na marginesie. To reguła jakości na wejściu.

- pole jest wymagane?
- jaki ma typ?
- jakie są dozwolone wartości?
- czy wartość ma sens w biznesie?
- co robimy z rekordami niezgodnymi?

Bez kontraktu pipeline działa, ale nie wie, co jest poprawne.

## ■ Kiedy kontrakt jest potrzebny

- gdy dane pochodzą z zewnętrznych źródeł
- gdy ktoś będzie je czytał i liczył KPI
- gdy statusy, daty, identyfikatory i waluty mogą mieć różne formaty
- gdy wynik ma wpływać na decyzje biznesowe

Bez kontraktu każda zmiana na wejściu może wyglądać jak „mały fix”, a w praktyce psuje metryki downstream.

## ■ Gdzie kontrakt jest zapisany

```
czytanie surowego payloadu  
-> walidacja na granicy  
-> zamiana na model domenowy  
-> kierowanie do zaakceptowanych/odrzuconych  
-> liczenie metryk tylko dla zaakceptowanych
```

To jest klucz do czystej logiki, dobrych metryk i zrozumiałego review.

## ■ Surowy dict kontra model domenowy

```
record = {  
    "order_id": "1001",  
    "status": " Completed ",  
    "total_amount": "120.50",  
}
```

**dict** jest elastyczny, ale nie jest kontraktem. Nie gwarantuje poprawności danych.

```
from dataclasses import dataclass  
  
@dataclass(frozen=True)  
class Order:  
    order_id: str  
    status: str  
    total_amount: float  
    source: str = "unknown"
```

Po walidacji taki obiekt jest gotowy do transformacji i agregacji.

```
from pydantic import BaseModel, Field,
field_validator

ALLOWED_STATUSES = {"completed", "cancelled", "
pending", "refunded"}

class OrderPayload(BaseModel):
    order_id: str = Field(min_length=1)
    status: str
    total_amount: float = Field(ge=0)
    source: str = "unknown"

    @field_validator("status")
    @classmethod
    def normalize_status(cls, value: str) -> str:
        normalized = value.strip().lower()
        if normalized not in ALLOWED_STATUSES:
            raise ValueError("invalid status")
        return normalized
```

Pydantic waliduje wejście zanim dane trafią do logiki biznesowej.  
To jest prawdziwy kontrakt danych.

## ■ Zaakceptowane kontra odrzucone

zaakceptowane -> liczymy KPI  
odrzucone -> zapisujemy przyczynę i metadane

Przykład payloadu odrzuconego:

```
{  
  "record": {"status": "done", "total_amount": "  
120.50"},  
  "reason": "missing order_id; invalid status",  
  "validation_stage": "boundary_contract",  
  "contract_version": "v1"  
}
```

Brak **order\_id** to nie „dziwny przypadek”. To jawny błąd kontraktu i sygnał do analizy źródła.

## ■ Co robić z odrzuconymi rekordami

- nie łączyć ich z głównym zestawem
- zapisywać przyczynę i źródło
- liczyć reject rate per source
- analizować trend w czasie
- skontaktować się z producentem, gdy wzorzec się zmienia

Odrzucone rekordy to nie „brudne dane”, które można po prostu zignorować. To jest operacyjny sygnał do poprawy źródła.

## ■ Schema drift

Schema drift to zmiana formatu danych po stronie producenta:

```
pole znika  
pole zmienia typ  
status dostaje nową wartość  
null zaczyna pojawiać się w wymaganym polu
```

Najgorsze są ciche zmiany, które trafiają do KPI bez ostrzeżenia.  
Wtedy analityk dostaje fałszywy obraz biznesu.

- 1) wykryj za pomocą walidacji i metryk odrzucania
- 2) sklasyfikuj typ zmiany
- 3) zaktualizuj kontrakt i testy
- 4) poinformuj właściciela źródła

Dobra polityka to nie „zawsze akceptuj”. To kontrolowany przepływ zmian z odpowiednim monitorowaniem.

## ■ Minimalne KPI jakości danych

```
accepted_count  
rejected_count  
reject_rate  
top_reject_reasons  
reject_rate_by_source  
contract_version
```

Jeśli **reject\_rate** skacze po wdrożeniu, traktujemy to jak incydent operacyjny, nie jako „normalny ruch”.

## ■ Co sprawdzamy w testach

```
missing required field  
wrong type  
invalid enum  
negative total_amount  
status normalization  
happy path  
schema drift scenario
```

Bez testów kontraktu nie ma zaufania do jakości danych. W data engineering testy są częścią kontroli jakości, nie dodatkiem.

## ■ Zasada projektu

```
Pydantic na granicy  
Dataclass po walidacji  
Rozdzielenie zaakceptowanych i odrzuconych  
Metryki tylko dla zaakceptowanych
```

To jest najważniejszy wzorzec tej lekcji. Z niego wyrasta bezpieczny, zrozumiały pipeline.

## Definition of Done

```
[ ] kontrakt jest jawnie zapisany w kodzie  
[ ] zaakceptowane i odrzucone rekordy są  
rozdzielone  
[ ] każdy odrzucony rekord ma przyczynę i metadane  
[ ] testy pokrywają drift, walidację i happy path  
[ ] metryki jakości są policzone i monitorowane
```

Po tej lekcji pipeline nie tylko działa. On chroni KPI przed złym wejściem i wyraźnie pokazuje, gdzie pojawił się błąd.

## ■ Odpowiedź na pytanie rekrutacyjne

Pytanie:

Jak radzisz sobie z kontraktami danych i schema drift w Pythonie?

Odpowiedź:

Definiuję kontrakt na granicy z pomocą Pydantic, waliduję wszystkie pola wejściowe i odrzucam niepoprawne rekordy z przyczyną. Tylko zaakceptowane rekordy stają się obiektami domenowymi i zasilają metryki biznesowe. Monitoruję reject rate oraz wersje kontraktu, żeby schema drift był wykrywany wcześniej i traktowany jak incydent jakości danych.

## Checklist do powtórki

- [ ] Czy potrafisz powiedzieć, czym jest kontrakt danych?
- [ ] Czy rozumiesz różnicę między surowym dict a modelem domenowym?
- [ ] Czy umiesz wyjaśnić, dlaczego metryki odrzucania są ważne?
- [ ] Czy rozpoznasz schema drift i wiesz, jak reagować?
- [ ] Czy potrafisz powiedzieć, co daje rozdzielenie zaakceptowanych i odrzuconych rekordów?

